

Task 1. Minimum passenger age

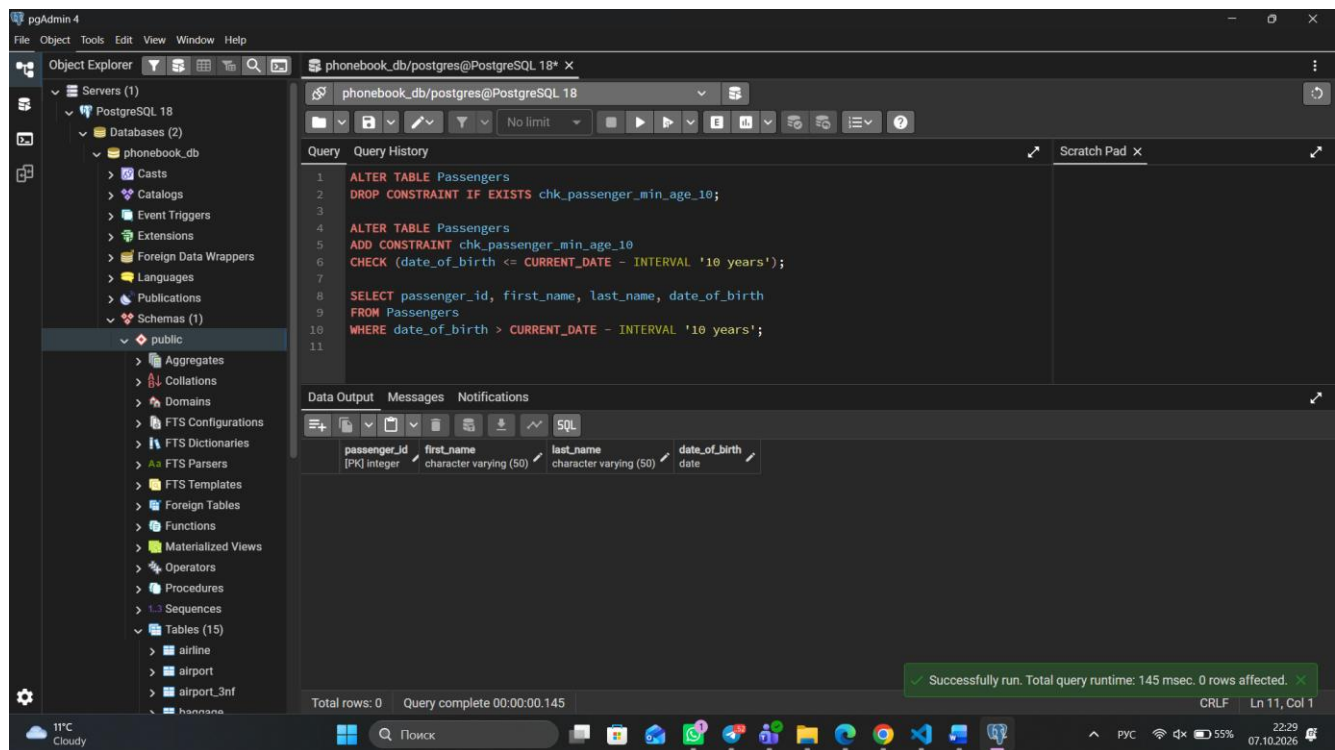
Requirement. Add a CHECK constraint so that every passenger is at least 10 years old.

SQL code

```
ALTER TABLE Passengers
DROP CONSTRAINT IF EXISTS chk_passenger_min_age_10;

ALTER TABLE Passengers
ADD CONSTRAINT chk_passenger_min_age_10
CHECK (date_of_birth <= CURRENT_DATE - INTERVAL '10 years');

SELECT passenger_id, first_name, last_name, date_of_birth
FROM Passengers
WHERE date_of_birth > CURRENT_DATE - INTERVAL '10 years';
```



Task 2. Booking price range

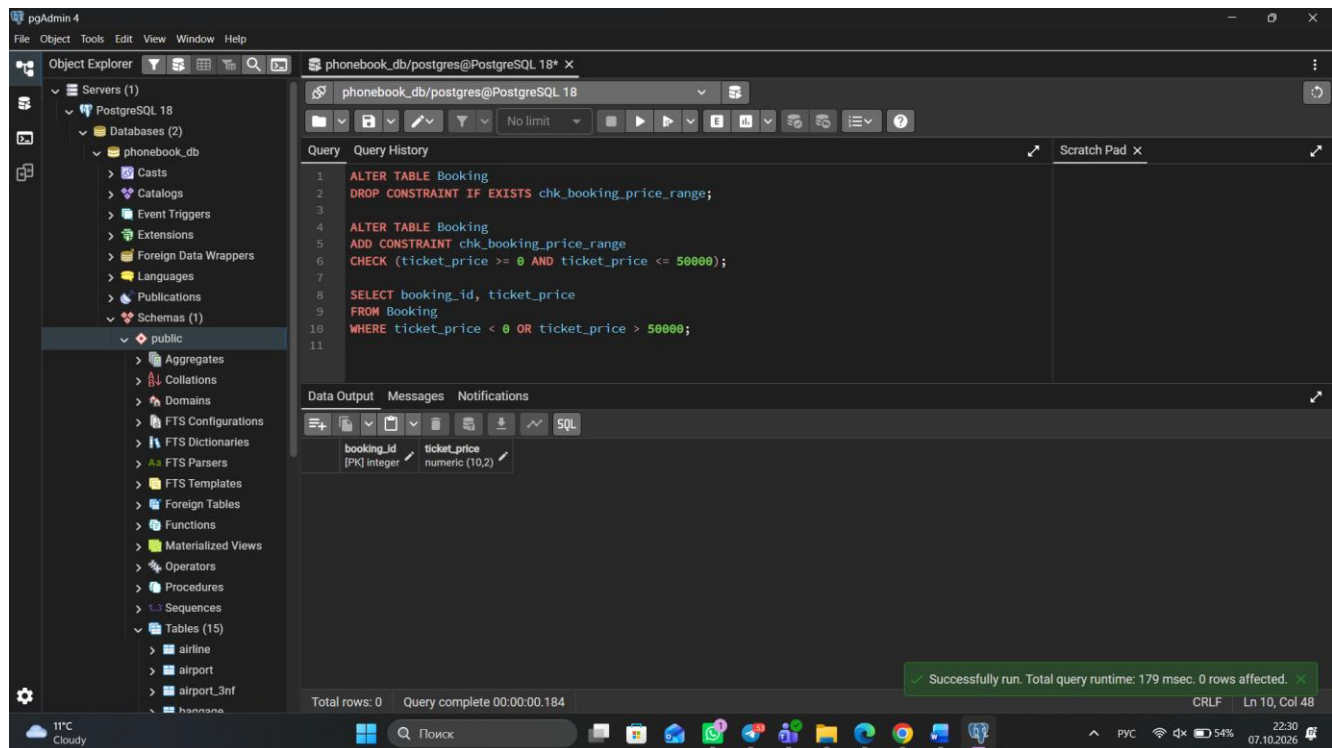
Requirement. Add a CHECK constraint for a practical ticket-price range from 0 to 50000 tg.

SQL code

```
ALTER TABLE Booking
DROP CONSTRAINT IF EXISTS chk_booking_price_range;

ALTER TABLE Booking
ADD CONSTRAINT chk_booking_price_range
CHECK (ticket_price >= 0 AND ticket_price <= 50000);

SELECT booking_id, ticket_price
FROM Booking
WHERE ticket_price < 0 OR ticket_price > 50000;
```



Task 3. Baggage weight

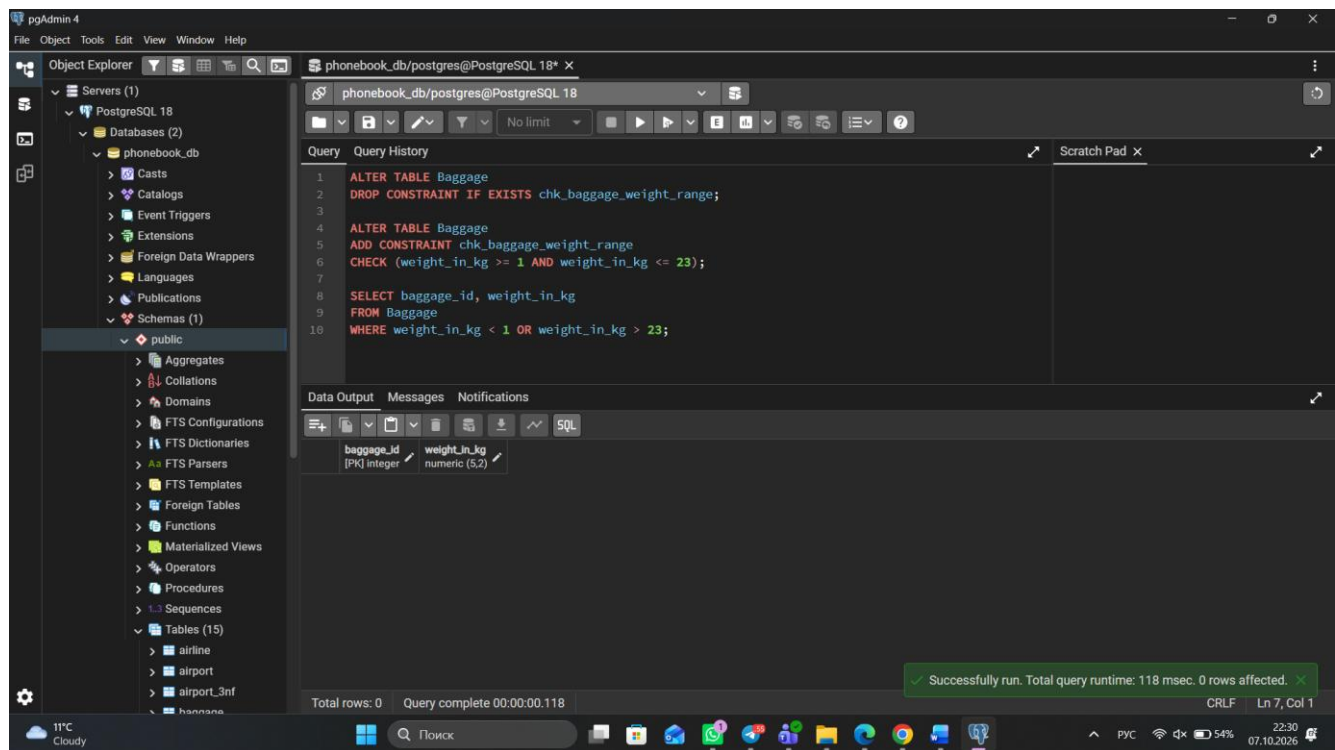
Requirement. Add a CHECK constraint to keep luggage weight between 1 and 23 kg.

SQL code

```
ALTER TABLE Baggage
DROP CONSTRAINT IF EXISTS chk_baggage_weight_range;

ALTER TABLE Baggage
ADD CONSTRAINT chk_baggage_weight_range
CHECK (weight_in_kg >= 1 AND weight_in_kg <= 23);

SELECT baggage_id, weight_in_kg
FROM Baggage
WHERE weight_in_kg < 1 OR weight_in_kg > 23;
```



Task 4. Airport name length

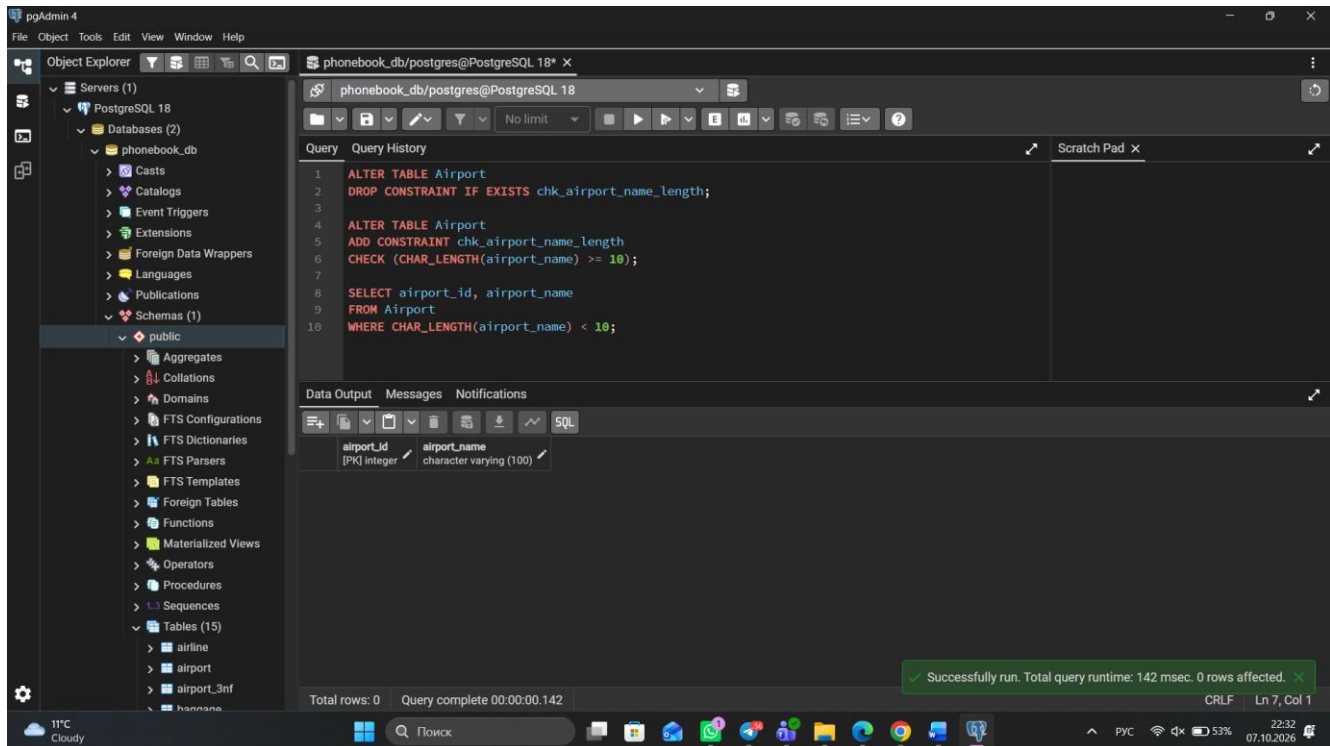
Requirement. Add a CHECK constraint so that airport_name contains at least 10 characters.

SQL code

```
ALTER TABLE Airport
DROP CONSTRAINT IF EXISTS chk_airport_name_length;

ALTER TABLE Airport
ADD CONSTRAINT chk_airport_name_length
CHECK (CHAR_LENGTH(airport_name) >= 10);

SELECT airport_id, airport_name
FROM Airport
WHERE CHAR_LENGTH(airport_name) < 10;
```



Task 5. UNIQUE constraints

Requirement. Add at least one UNIQUE constraint to every table in the database.

SQL code

```
ALTER TABLE Airline
DROP CONSTRAINT IF EXISTS uq_airline_code;

ALTER TABLE Airline
ADD CONSTRAINT uq_airline_code UNIQUE (airline_code);

ALTER TABLE Airport
DROP CONSTRAINT IF EXISTS uq_airport_name;

ALTER TABLE Airport
ADD CONSTRAINT uq_airport_name UNIQUE (airport_name);

ALTER TABLE Passengers
DROP CONSTRAINT IF EXISTS uq_passenger_passport;

ALTER TABLE Passengers
ADD CONSTRAINT uq_passenger_passport UNIQUE (passport_number);

ALTER TABLE Flights
DROP CONSTRAINT IF EXISTS uq_flight_number;

ALTER TABLE Flights
ADD CONSTRAINT uq_flight_number UNIQUE (flight_number);

ALTER TABLE Booking
DROP CONSTRAINT IF EXISTS uq_booking_passenger_flight;

ALTER TABLE Booking
ADD CONSTRAINT uq_booking_passenger_flight UNIQUE (passenger_id, flight_id);

ALTER TABLE Booking_flight
DROP CONSTRAINT IF EXISTS uq_booking_flight_pair;

ALTER TABLE Booking_flight
ADD CONSTRAINT uq_booking_flight_pair UNIQUE (booking_id, flight_id);

ALTER TABLE Baggage
DROP CONSTRAINT IF EXISTS uq_baggage_booking_time;

ALTER TABLE Baggage
ADD CONSTRAINT uq_baggage_booking_time UNIQUE (booking_id, created_at);

ALTER TABLE Baggage_check
DROP CONSTRAINT IF EXISTS uq_baggage_check_booking_passenger;

ALTER TABLE Baggage_check
ADD CONSTRAINT uq_baggage_check_booking_passenger UNIQUE (booking_id, passenger_id);

ALTER TABLE Boarding_pass
DROP CONSTRAINT IF EXISTS uq_boarding_pass_booking;

ALTER TABLE Boarding_pass
ADD CONSTRAINT uq_boarding_pass_booking UNIQUE (booking_id);

ALTER TABLE Security_check
DROP CONSTRAINT IF EXISTS uq_security_check_passenger;

ALTER TABLE Security_check
ADD CONSTRAINT uq_security_check_passenger UNIQUE (passenger_id);

SELECT conname, conrelid::regclass AS table_name
FROM pg_constraint
WHERE contype = 'u'
ORDER BY table_name, conname;
```


Task 6. Gender and minimum age

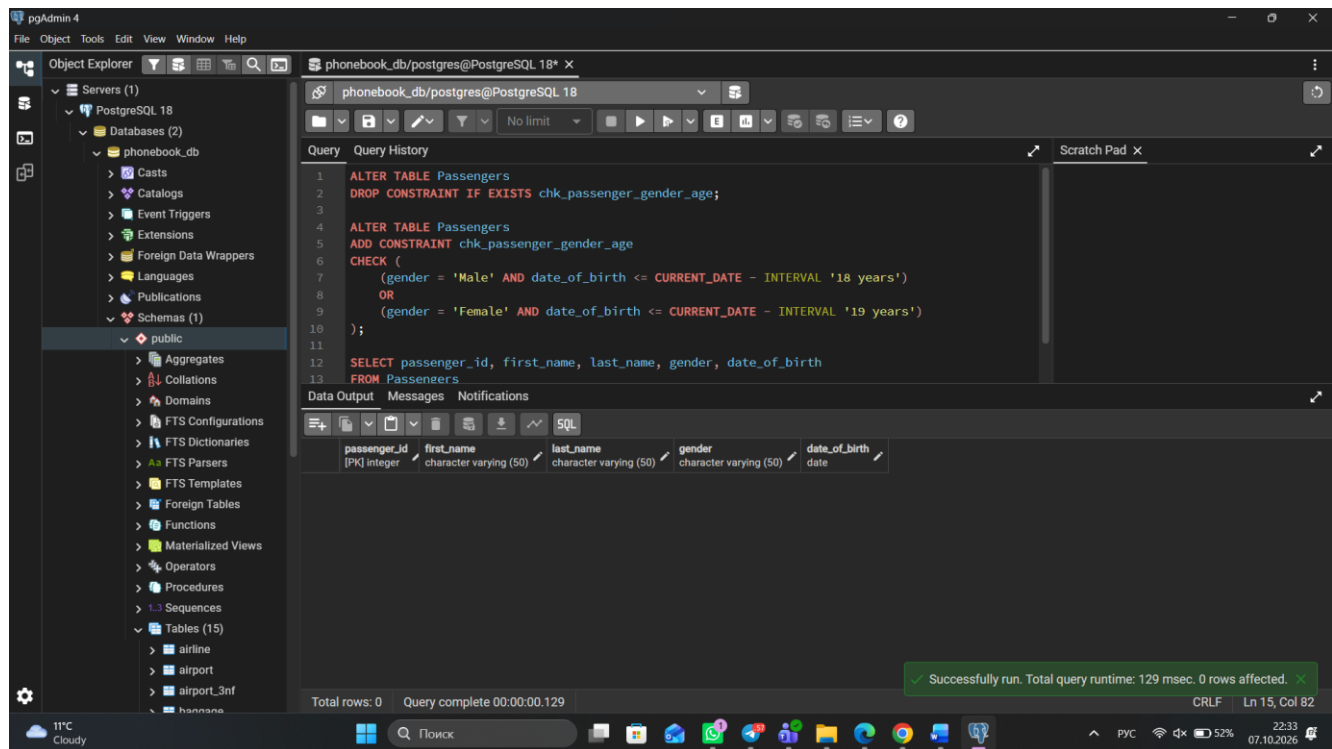
Requirement. Ensure that male passengers are at least 18 years old and female passengers are at least 19 years old.

SQL code

```
ALTER TABLE Passengers
DROP CONSTRAINT IF EXISTS chk_passenger_gender_age;

ALTER TABLE Passengers
ADD CONSTRAINT chk_passenger_gender_age
CHECK (
    (gender = 'Male' AND date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
    OR
    (gender = 'Female' AND date_of_birth <= CURRENT_DATE - INTERVAL '19 years')
);

SELECT passenger_id, first_name, last_name, gender, date_of_birth
FROM Passengers
WHERE (gender = 'Male' AND date_of_birth > CURRENT_DATE - INTERVAL '18 years')
    OR (gender = 'Female' AND date_of_birth > CURRENT_DATE - INTERVAL '19 years');
```



Task 7. Citizenship and minimum age

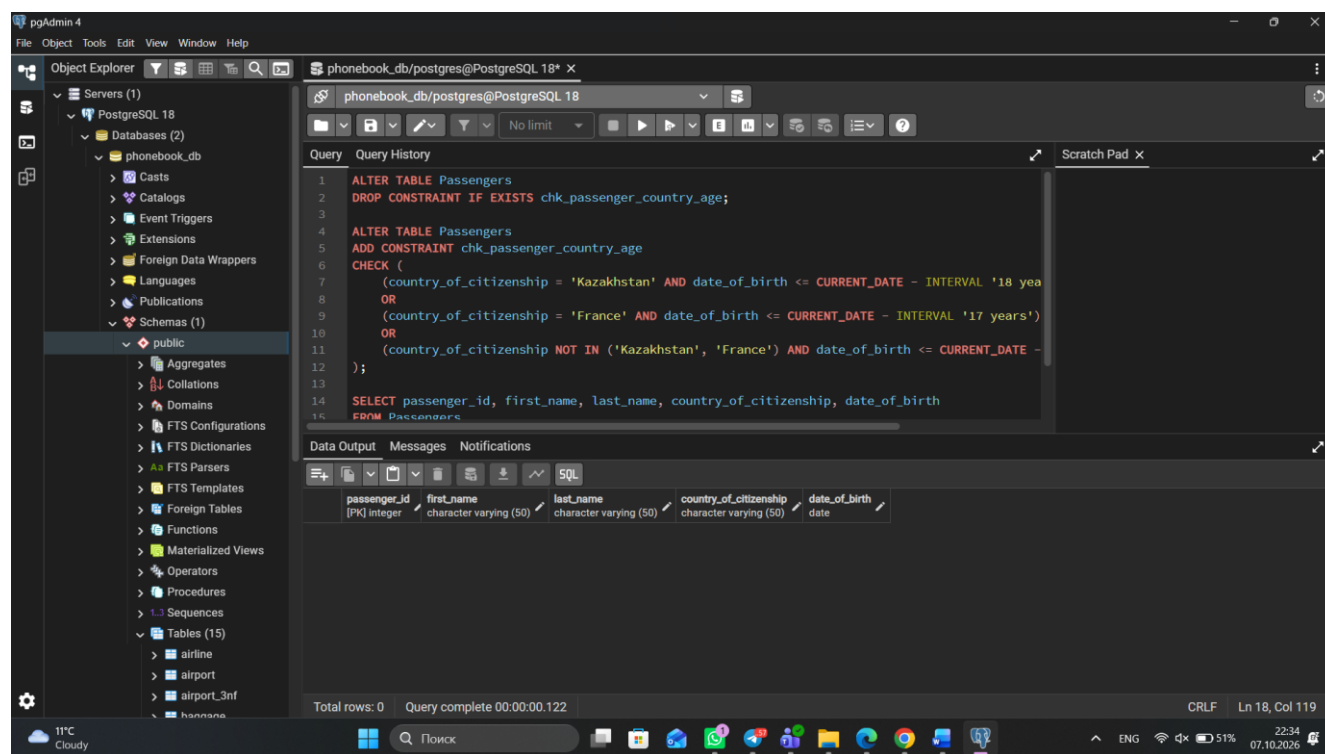
Requirement. Apply age rules by country of citizenship: Kazakhstan - 18 years, France - 17 years, other countries - 19 years.

SQL code

```
ALTER TABLE Passengers
DROP CONSTRAINT IF EXISTS chk_passenger_country_age;

ALTER TABLE Passengers
ADD CONSTRAINT chk_passenger_country_age
CHECK (
    (country_of_citizenship = 'Kazakhstan' AND date_of_birth <= CURRENT_DATE - INTERVAL '18 years')
    OR
    (country_of_citizenship = 'France' AND date_of_birth <= CURRENT_DATE - INTERVAL '17 years')
    OR
    (country_of_citizenship NOT IN ('Kazakhstan', 'France') AND date_of_birth <= CURRENT_DATE - INTERVAL '19
years')
);

SELECT passenger_id, first_name, last_name, country_of_citizenship, date_of_birth
FROM Passengers
WHERE (country_of_citizenship = 'Kazakhstan' AND date_of_birth > CURRENT_DATE - INTERVAL '18 years')
    OR (country_of_citizenship = 'France' AND date_of_birth > CURRENT_DATE - INTERVAL '17 years')
    OR (country_of_citizenship NOT IN ('Kazakhstan', 'France') AND date_of_birth > CURRENT_DATE - INTERVAL
'19 years');
```



Task 8. Ticket discount

Requirement. Add ticket_discount and validate a 5% discount after 2024-01-01 and a 10% discount before 2024-01-01.

SQL code

```
ALTER TABLE Booking
ADD COLUMN IF NOT EXISTS ticket_discount DECIMAL(10, 2) NOT NULL DEFAULT 0;

UPDATE Booking
SET ticket_discount = CASE
    WHEN created_at::date > DATE '2024-01-01' THEN ROUND(ticket_price * 0.05, 2)
    WHEN created_at::date < DATE '2024-01-01' THEN ROUND(ticket_price * 0.10, 2)
    ELSE 0.00
END;

ALTER TABLE Booking
DROP CONSTRAINT IF EXISTS chk_booking_ticket_discount;

ALTER TABLE Booking
ADD CONSTRAINT chk_booking_ticket_discount
CHECK (
    (created_at::date > DATE '2024-01-01' AND ticket_discount = ROUND(ticket_price * 0.05, 2))
    OR
    (created_at::date < DATE '2024-01-01' AND ticket_discount = ROUND(ticket_price * 0.10, 2))
    OR
    (created_at::date = DATE '2024-01-01' AND ticket_discount = 0.00)
);

SELECT booking_id, ticket_price, created_at, ticket_discount
FROM Booking
ORDER BY created_at;
```

The screenshot shows the pgAdmin 4 interface with the following components:

- Object Explorer:** Displays the database structure for 'phonebook_db' under 'PostgreSQL 18'. The 'public' schema is selected, showing various objects like Aggregates, Collations, Domains, etc.
- Query Editor:** Contains the SQL code from the previous block, executed against the 'phonebook_db/postgres@PostgreSQL 18' connection.
- Data Output:** Shows the result of the SELECT query, displaying 12 rows of booking data with columns: booking_id, ticket_price, created_at, and ticket_discount.
- Messages:** A green message box at the bottom right states: 'Successfully run. Total query runtime: 116 msec. 12 rows affected.'

booking_id	ticket_price	created_at	ticket_discount
201	30000.00	2023-12-15 11:20:00	3000.00
5001	45000.00	2026-09-15 11:00:00	2250.00
5002	44000.00	2026-09-15 11:05:00	2200.00
5003	38000.00	2026-09-15 11:10:00	1900.00
5004	36000.00	2026-09-15 11:12:00	1800.00
5005	32000.00	2026-09-15 11:15:00	1600.00
5006	30000.00	2026-09-15 11:17:00	1500.00
200	42000.00	2026-09-17 11:20:00	2100.00

Task 9. Default booking status

Requirement. Change the booking status so that its default value is 'Confirmed', then insert a booking without specifying status.

SQL code

```
ALTER TABLE Booking
ALTER COLUMN status SET DEFAULT 'Confirmed';

-- Удаляем запись с ID 6001, если она уже существует
DELETE FROM Booking WHERE booking_id = 6001;

INSERT INTO Booking (
    booking_id,
    flight_id,
    passenger_id,
    booking_platform,
    created_at,
    updated_at,
    ticket_price,
    ticket_discount
)
VALUES (
    6001,
    1005,
    4,
    'Web',
    CURRENT_TIMESTAMP,
    CURRENT_TIMESTAMP,
    40000.00,
    2000.00
);

SELECT booking_id, status, ticket_price, ticket_discount, created_at
FROM Booking
WHERE booking_id = 6001;
```

The screenshot shows the pgAdmin 4 interface with the following components:

- Object Explorer:** Displays the database structure for 'phonebook_db' under 'PostgreSQL 18'. The 'public' schema is selected, showing various objects like Aggregates, Collations, Domains, etc.
- Query Editor:** Contains the SQL code from the previous block. The query history shows the execution of the INSERT and SELECT statements.
- Data Output:** Displays the result of the SELECT query. It shows a single row with the following data:

booking_id [PK] integer	status character varying (50)	ticket_price numeric (10,2)	ticket_discount numeric (10,2)	created_at timestamp without time zone
1	6001	Confirmed	40000.00	2026-10-07 22:36:15.92123

At the bottom, a status bar indicates: 'Total rows: 1', 'Query complete 00:00:00.108', and a green message: 'Successfully run. Total query runtime: 108 msec. 1 rows affected.'

Task 10. Automatic creation date

Requirement. Modify an appropriate date column so that a new record automatically receives the current date/time.

SQL code

```
ALTER TABLE Booking
ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP;

-- Удаление для безопасного повторного выполнения
DELETE FROM Booking WHERE booking_id = 6002;

INSERT INTO Booking (
    booking_id,
    flight_id,
    passenger_id,
    booking_platform,
    updated_at,
    status,
    ticket_price,
    ticket_discount
)
VALUES (
    6002,
    1005,
    5,
    'Mobile',
    CURRENT_TIMESTAMP,
    'Confirmed',
    42000.00,
    2100.00
);

SELECT booking_id, created_at, CURRENT_DATE AS current_date, status
FROM Booking
WHERE booking_id = 6002;
```

The screenshot shows the pgAdmin 4 interface with the following components:

- Object Explorer:** Displays the database structure for 'phonebook_db' under 'PostgreSQL 18'. The 'public' schema is selected, showing various objects like Aggregates, Collations, Domains, etc.
- Query Editor:** Contains the SQL code from the previous block. The query is executed, and the results are displayed in the 'Data Output' pane.
- Data Output:** Shows a single row of data for the 'Booking' table with the following values:

booking_id	created_at	current_date	status
6002	2026-10-07 22:39:01.514368	2026-10-07	Confirmed
- Messages:** A green message at the bottom indicates 'Successfully run. Total query runtime: 130 msec. 1 rows affected.'
- Status Bar:** Shows 'Total rows: 1' and 'Query complete 00:00:00.130'.